Лабораторна робота №13

Тема: Створення FastAPI додатку з використанням Ormar моделі з використанням зовнішніх ключів
Мета: Освоєння створення моделей з використанням зовнішніх ключів у Ormar та їх використання у FastAPI додатку для взаємодії з базою даних.

Хід роботи

Для цієї лабораторної роботи необхідно виконати попередню лабораторну роботу, оскільки вона буде розширювати та доповнювати результати або методи, що були використані у попередній роботі.

У файлі *models.py* додамо модель Dog:

```
import ormar

from database import database, metadata


class Human(ormar.Model):
    id: int = ormar.BigInteger(primary_key=True, nullable=False)
    name: str = ormar.String(max_length=50)
    surname: str = ormar.String(max_length=50)
    age: int = ormar.Integer(minimum=0)

    class Meta(ormar.ModelMeta):
        tablename = "human"
        database = database
        metadata = metadata


class Dog(ormar.Model):
    id: int = ormar.BigInteger(primary_key=True)
    name: str = ormar.String(max_length=50)
    age: int = ormar.Integer(minimum=0)
    is_male: bool = ormar.Boolean()
    owner: Human | dict | None = ormar.ForeignKey(Human)

    class Meta(ormar.ModelMeta):
        tablename = "dog"
        database = database
        metadata = metadata
```

Модель Dog описує сутність "Собака" у системі. Вона має поля id, name, age, is_male та owner, які відповідають за ідентифікацію, ім'я, вік, стать та власника собаки відповідно.

Додамо до проекту файл *schemas.py*:

```
from pydantic import BaseModel, Field


class DogCreate(BaseModel):
    name: str = Field(max_length=50)
    age: int = Field(ge=0)
    is_male: bool
    owner: int | None


class DogUpdate(BaseModel):
    name: str = Field(None, max_length=50)
    age: int = Field(None, ge=0)
    is_male: bool = None
    owner: int | None = None


class HumanCreate(BaseModel):
    name: str = Field(max_length=50)
    surname: str = Field(max_length=50)
    age: int = Field(ge=0)


class HumanUpdate(BaseModel):
    name: str = Field(None, max_length=50)
    surname: str = Field(None, max_length=50)
    age: int = Field(None, ge=0)
```

В подальшому, ці схеми будуть використані для створення endpoint'ів. Зверніть увагу: в update-моделях всі поля по замовчуванню None - так ми реалізуємо PATCH-методи.

Оновимо файл *main.py:*

```
import sqlite3

from fastapi import FastAPI, Response
from fastapi.exceptions import HTTPException
from ormar.exceptions import NoMatch

from database import database, engine, metadata
from models import Dog, Human
from schemas import DogCreate, DogUpdate, HumanCreate, HumanUpdate

app = FastAPI()

app.state.database = database


@app.on_event("startup")
```

```python
async def startup():
    database_ = app.state.database
    if not database_.is_connected:
        await database_.connect()
    metadata.create_all(engine)


@app.on_event("shutdown")
async def shutdown():
    database_ = app.state.database
    if database_.is_connected:
        await database_.disconnect()


@app.get("/human/")
async def get_humans():
    humans = await Human.objects.all()
    return humans


@app.get("/human/{id}")
async def get_human_by_id(id: int):
    try:
        human = await Human.objects.get(id=id)
    except NoMatch:
        raise HTTPException(status_code=404, detail=f"Human with id {id} not found!")
    return human


@app.patch("/human/{id}")
async def update_human(id: int, human: HumanUpdate):
    try:
        db_human = await Human.objects.get(pk=id)
    except NoMatch:
        raise HTTPException(status_code=404, detail=f"Human with id {id} not found!")
    await db_human.update(**human.dict(exclude_unset=True))
    return db_human


@app.delete("/human/{id}")
async def delete_human(id: int):
    try:
        await Human.objects.delete(id=id)
    except NoMatch:
        raise HTTPException(status_code=404, detail=f"Human with id {id} not found!")
    except sqlite3.IntegrityError as e:
        raise HTTPException(status_code=409, detail=str(e))
    return Response(status_code=204)


@app.get("/dog/")
async def get_dogs():
    dogs = await Dog.objects.all()
    return dogs


@app.get("/dog/{id}")
async def get_dog_by_id(id: int):
    try:
        dog = await Dog.objects.get(id=id)
```

```
    except NoMatch:
        raise HTTPException(status_code=404, detail=f"Dog with id {id} not found!")
    return dog


@app.post("/dog/")
async def create_dog(dog: DogCreate):
    try:
        return await Dog(**dog.dict()).save()
    except sqlite3.IntegrityError as e:
        raise HTTPException(status_code=404, detail=str(e))


@app.patch("/dog/{id}")
async def update_dog(id: int, dog: DogUpdate):
    try:
        return await Dog.objects.update_or_create(pk=id, **dog.dict(exclude_unset=True))
    except NoMatch:
        raise HTTPException(status_code=404, detail=f"Dog with id {id} not found!")


@app.delete("/dog/{id}")
async def delete_dog(id: int):
    try:
        await Dog.objects.delete(id=id)
    except NoMatch:
        raise HTTPException(status_code=404, detail=f"Dog with id {id} not found!")
    except sqlite3.IntegrityError as e:
        raise HTTPException(status_code=409, detail=str(e))
    return Response(status_code=204)


@app.post("/human/")
async def create_human(human: HumanCreate):
    try:
        return await Human(**human.dict()).save()
    except sqlite3.IntegrityError as e:
        raise HTTPException(status_code=404, detail=str(e))
```

Ендпоінти /human/ дозволяють отримати список людей, створити нового користувача, а також оновити чи видалити існуючого. Так само, ендпоінти /dog/ надають можливість працювати з собаками, включаючи отримання списку, створення, оновлення та видалення окремих екземплярів собак.

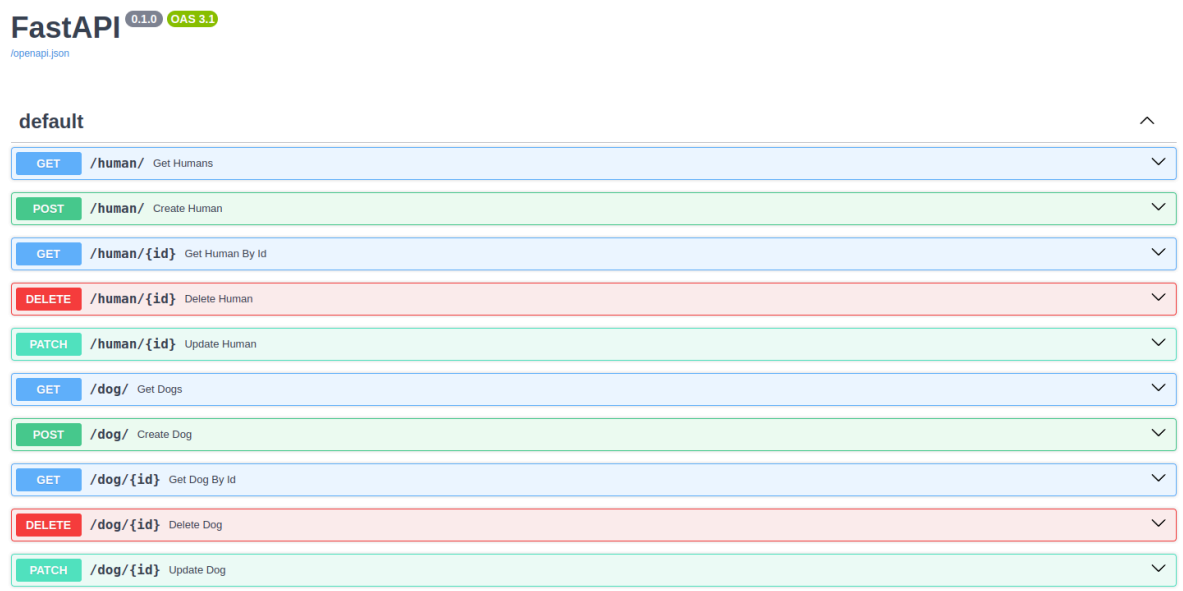
У вказаних ендпоінтах використовуються схеми для приховування непотрібних полів або для імплементації функціоналу оновлення конкретних полів.

Наприклад, схема DogUpdate в ендпоінті оновлення собаки /dog/{id}

дозволяє користувачу оновлювати лише ті поля собаки, які він бажає, за допомогою вказання лише потрібних полів у відповідному PATCH-запиті. Це виконується за допомогою параметра exclude_unset=True у методі dog.dict(), що вказує на те, щоб у вихідному словнику не були включені поля зі значенням None.

Також, схема DogCreate при створенні нової собаки у ендпоінті /dog/ дозволяє користувачу вказати лише обов'язкові поля (такі як ім'я, вік та стать), використовуючи обмеження Field(max_length=50) для імені та Field(ge=0) для віку, а також параметр owner: int | None, який дозволяє зберегти ідентифікатор власника собаки або значення None, якщо власник не вказаний.

Запустимо додаток та оглянемо наш додаток у Swagger:



Усі endpoint-и завантажились, тепер їх можна протестувати.

Завдання на самостійну роботу:

Ваше завдання - розширити раніше розроблений додаток з ЛР, додавши моделі з зовнішніми ключами, а також розробити CRUD для цих моделей.

Для виконання цього завдання, слід дотримуватися наступних кроків:

1. Додати нові моделі з зовнішніми ключами: Спочатку потрібно визначити нові моделі, які будуть мати зовнішні ключі для зв'язку з іншими моделями у базі даних.
2. Додати схеми для нових моделей: Створіть схеми Pydantic для нових моделей, які будуть використовуватися в ендпоінтах для валідації вхідних даних та серіалізації вихідних даних.
3. Розробити CRUD операції для нових моделей: Створіть ендпоінти для створення, читання, оновлення та видалення об'єктів нових моделей.
4. Додати зв'язки між моделями: Визначте відповідні зв'язки між новими моделями та існуючими моделями, які вже є в додатку.
5. Перевірка правильності даних: Додайте перевірку вхідних даних на відповідність вимогам моделей та забезпечте їх цілісність.
6. Тестування: Переконайтеся, що всі нові функції працюють правильно, та виконайте тестування для перевірки функціональності додатку.

Контрольні запитання:

1. Як в Ormar використовуються зовнішні ключі (foreign keys) для встановлення зв'язків між моделями?
2. Що таке відношення "багато-до-багатьох" (many-to-many) у зв'язках між моделями бази даних, і як це реалізовано в Ormar?
3. Навіщо потрібні додаткові pydantic схеми, якщо у нас є ormar моделі?
4. Які є загальні підходи до валідації даних у схемах Pydantic?
5. Які стратегії обробки помилок варто використовувати при роботі з базами даних через Ormar в FastAPI додатках?